

Cooking Buddy

AI-Powered Interactive Cooking Assistant

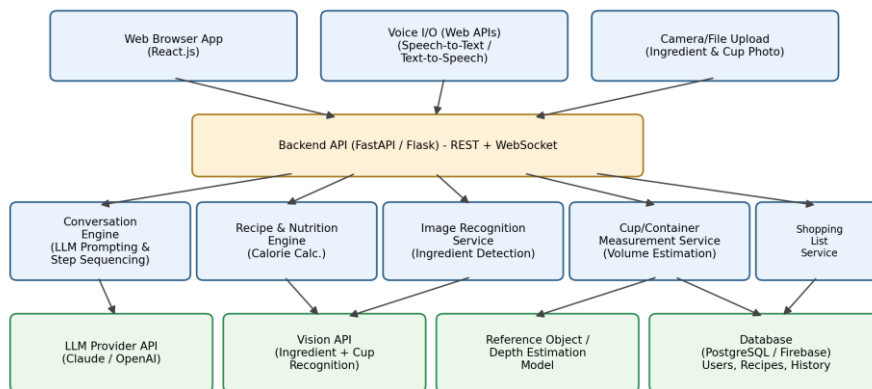
1. Overview

This project proposes an AI-powered cooking companion delivered as a web application, going beyond a typical recipe generator. The assistant holds a natural conversation with the user through the browser: it asks about dietary preferences, time available, and health goals, recommends a suitable recipe, states its total calorie count up front, then guides the user through cooking one step at a time, waiting for confirmation before moving on. If the user has no standard measuring cups or a scale, they can point the camera at any container they do have; the system estimates its volume and automatically rescales the recipe's quantities to match. The website also supports voice interaction, recognizes ingredients from a photo, suggests substitutions, and builds a shopping list for missing items — making the AI feel like an interactive cooking coach rather than a passive lookup tool.

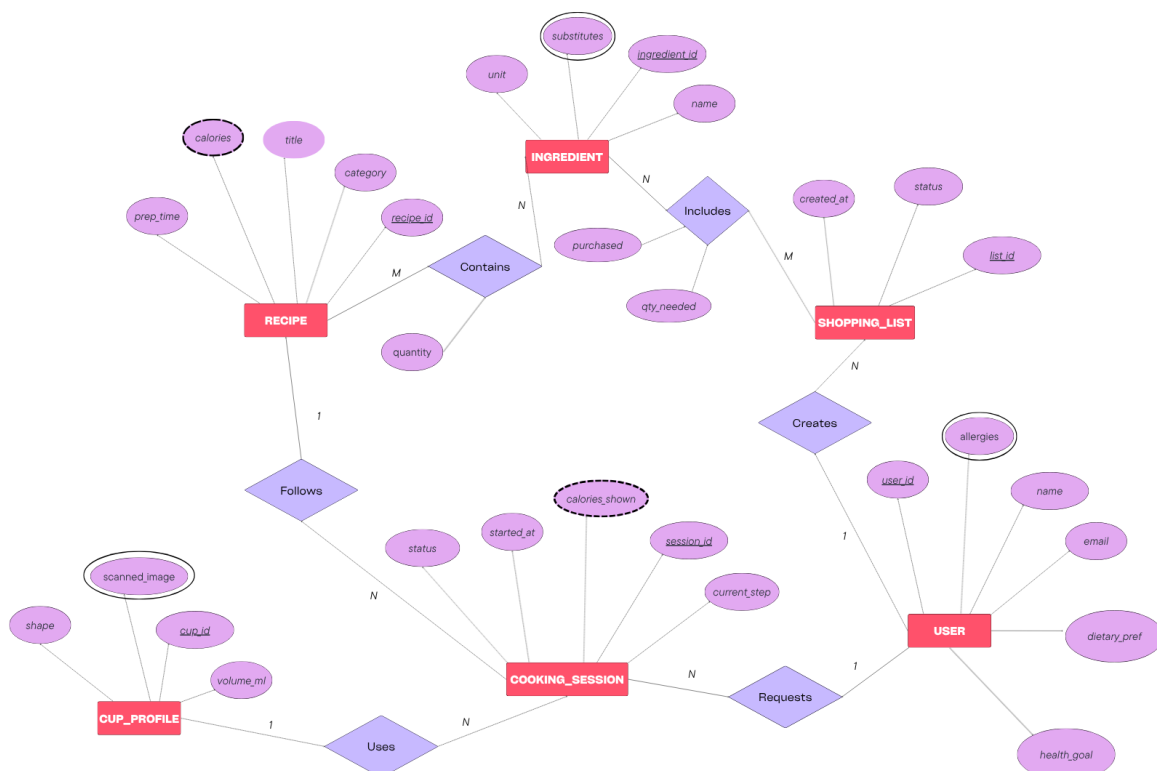
2. Challenges Intended to Solve Using AI Integration

- Decision fatigue: users often do not know what to cook; the assistant proactively suggests options based on goals, time, and available ingredients.
- Lack of real-time guidance: printed or static recipes cannot answer follow-up questions or adapt pace; the AI provides step-by-step, conversational guidance.
- Hands-busy interaction: touching a screen while cooking is impractical; voice commands solve this naturally.
- Ingredient gaps: users frequently discover missing items mid-recipe; smart substitution and shopping-list generation reduce interruption.
- Generic recommendations: most recipe apps ignore personal health goals; this assistant personalizes suggestions to weight-management, allergies, and family size.
- No standard measuring tools: many home cooks lack cups or a scale; the camera estimates a container's volume and rescales the recipe to match.
- Hidden nutritional impact: the assistant shows the total calorie estimate at the start of the recipe, before the user commits to cooking it.

3. System Architecture Diagram



4. System ER Diagram



5. Functional and Non-Functional Requirements

Functional Requirements	Non-Functional Requirements
Conduct conversational intake (dietary needs, allergies, servings, time available)	Usability: guided, step-by-step UI understandable by novice cooks
Recommend recipes matched to user goals and available time	Performance: AI response within 2-3 seconds per turn
Recognise ingredients from an uploaded/captured photo	Reliability: session state persists if connection drops mid-cook
Guide cooking one step at a time, waiting for user confirmation	Scalability: backend supports concurrent multi-user sessions
Suggest ingredient substitutions on request	Security: user data and health info stored securely, access-controlled
Support voice commands (next step, repeat, set timer, ask quantity)	Accuracy: nutrition/health suggestions framed responsibly, not medical advice
Generate a shopping list for missing ingredients	Portability: fully responsive, works across desktop and mobile browsers
Track cooking history and personalize future suggestions	Maintainability: modular services for AI, recipes, and voice I/O
Estimate a scanned cup/container's volume from a photo and rescale recipe quantities accordingly	Accessibility: works without any standard measuring tools or a kitchen scale
Display total estimated calories for the recipe before cooking begins	Transparency: nutrition estimates clearly labelled as approximate, not medical-grade

6. Technology Details

Frontend

React.js single-page website with a responsive, conversational chat-style UI, step cards, and browser-based voice controls (works on desktop and mobile browsers, no app install required).

Backend

Python with FastAPI, exposing REST endpoints and a WebSocket channel for real-time step-by-step interaction.

AI / Conversation Layer

Claude / OpenAI LLM API for dialogue management, recipe reasoning, and personalized recommendations, orchestrated through prompt templates and session context.

Computer Vision

Vision API for identifying ingredients from a photo, plus a reference-object / depth-estimation model to estimate a scanned container's volume in milliliters and rescale ingredient quantities proportionally.

Voice

Web Speech API (browser-native) or Whisper for speech-to-text; browser-native or cloud text-to-speech for spoken responses.

Database

PostgreSQL or Firebase to store user profiles, recipes, ingredient data, cooking sessions, and shopping lists.

Hosting

Cloud deployment (e.g. Render/Railway or Firebase Hosting) with the LLM and vision calls made server-side to protect API keys.

7. Conclusion

This project combines conversational AI, computer vision, voice interaction, and personalization into a single, practical cooking assistant. It goes beyond "enter ingredients, get a recipe" by guiding the user through cooking, adapting to health goals, estimating calories up front, and removing the need for standard measuring tools by reading quantities from whatever container is on hand. The scope suits a two-person academic project, with clear modules (conversation engine, recipe/nutrition logic, vision recognition, cup/volume estimation, voice I/O) developed and demonstrated incrementally.

8. GitHub Link

<https://github.com/teamkitchen711/AI-Cooking-Assistant>